Operations on Singly Linked List

Operations in Singly Linked List

In a singly linked list, each element (known as a node) contains a data value and a reference (or link) to the next node in the sequence. Given this basic structure, let's explore some common operations:

1. Insert

Insertion in a singly linked list can be of three types:

- **At the Beginning - O(1)**: Add a new node before the current head of the list.
- **At the End - O(n)**: Traverse the list to the last node and add the new node after it.
- **After a Given Node - O(n)**: Traverse the list to the desired node and insert the new node after it.

2. Delete

To delete a node:

- **From the Beginning - O(1)**: Set the second node as the new head.
- **From the End - O(n)**: Traverse the list and remove the reference to the last node.
- **A Given Node - O(n)**: Traverse to the node before the one to delete, then remove the reference to the node-to-delete.

3. Search - O(n)

Traverse the list from the head, comparing each node's data with the search value until a match is found or the end of the list is reached.

Now, let's write the code for these operations.

Java

```java
// Class to represent a ListNode in the linked list
// class ListNode {
//     int val; // Data of the node
//     ListNode next; // Reference to the next node in the list
//     // Constructor to initialize the node with val and set the next reference to null
```

```
//     ListNode(int d) { val = d; next = null; }
// }

// Class to represent a Singly Linked List and perform various operations
class Solution {
    ListNode head; // head of the list, initially null

    // Inserts a new ListNode at the front of the list.
    void insert(int val) {
        ListNode newNode = new ListNode(val); // Create a new node with the given val
        newNode.next = head; // Set the next reference of the new node to the current head
        head = newNode; // Update the head of the list to the new node
    }

    // Inserts a new node after the given prev_node.
    void insertAfter(ListNode prev_node, int val) {
        // Check if the given prev_node is null
        if (prev_node == null) {
            System.out.println("The given previous node cannot be null");
```

Run Code Result | Stdout

You must run your code first

Execute

← Previous

Introduction To LinkedList

Next →

Operations On Doubly Linked List

Mark as Completed